

Lógica y Representación 1

Conceptos básicos de Python

Ingeniería de Sistemas

Universidad de Antioquia

Método de Polya - Resumen



1

Entender el problema

2

Diseñar un plan

3

Implementar el plan

4

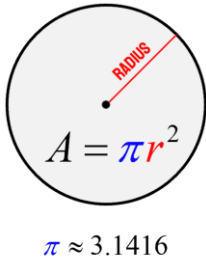
Revisar el plan

Resumiendo del proceso de diseño del algoritmos

Ejemplo:

Hacer un algoritmo que calcule el área de un círculo

1



2

Análisis:

Datos de entrada: radio

Datos de salida: área del círculo

Constantes:

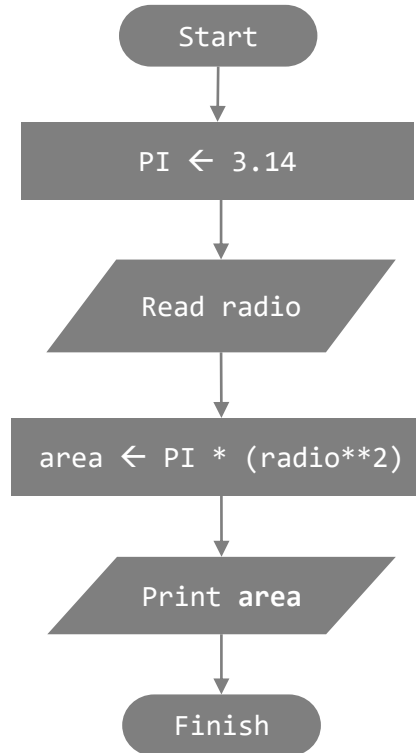
- **Numero PI:** $PI = 3.14$ (Real)

Definición de variables:

- **radio:** radio (Real)
- **area:** radio (Real)

¿Proceso?: Elevar el radio al cuadrado y multiplicarlo por π

3



Inicio

$PI = 3.14$

Leer(radio)

$area = PI * (radio**2)$

Escribir(area)

Fin

4

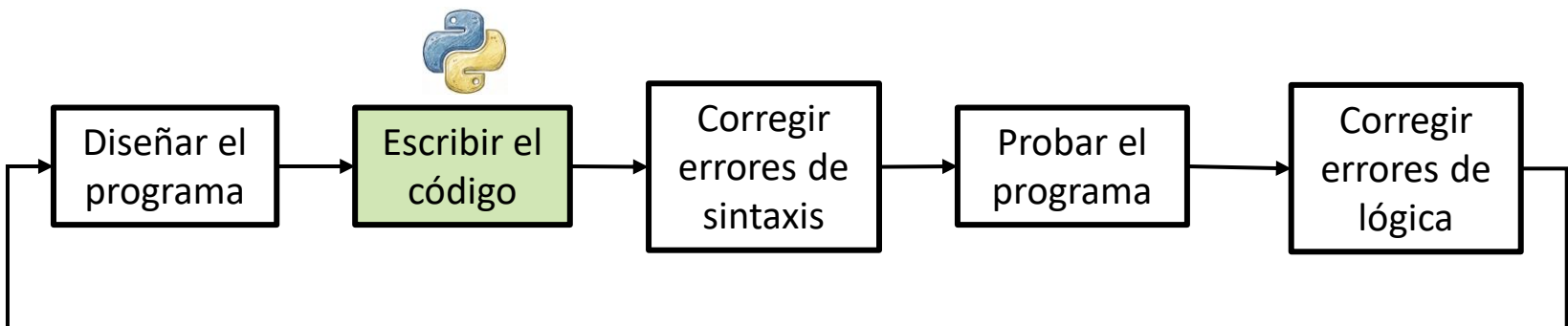
Entradas		Salidas	
radio		area	
10		314	
2.6		21.237	

↘radio	↗area
?	?
↘10	↗314

↘radio	↗area
?	?
↘2.6	↗21.226

Diseño de programas

- Los programas deben diseñarse cuidadosamente antes de ser escritos. Durante el proceso de diseño, los programadores utilizan herramientas como pseudocódigo y diagramas de flujo para crear modelos de los programas.
- **El ciclo de desarrollo de un programa:**
 1. Diseñar el programa
 2. Escribir el código
 3. Corregir errores de sintaxis
 4. Probar el programa
 5. Corregir errores de lógica



Codificación de algoritmos en Python

Ejemplo: Hacer un algoritmo que calcule el área de un círculo

```
# Inicialización de variables
PI = 3.14

# Entrada de datos
radio = float(input("Ingrese el radio del círculo: "))

# Calculo del area
area = PI * radio ** 2

# Despligue del area
print(f"El area del círculo de radio {radio} es: {area}")
```



[link](#)

Entradas		Salidas	
radio		area	
10		314	
2.6		21.237	

```
Terminal 1/A x
In [10]: runfile('D:/UdeA/2026-2/repos/logica1-20262/clases/3/ejemplo3-circulo.py',
wdir='D:/UdeA/2026-2/repos/logica1-20262/clases/3')
Ingrese el radio del círculo: 10
El area del círculo de radio 10.0 es: 314.0

In [11]: runfile('D:/UdeA/2026-2/repos/logica1-20262/clases/3/ejemplo3-circulo.py',
wdir='D:/UdeA/2026-2/repos/logica1-20262/clases/3')
Ingrese el radio del círculo: 2.6
El area del círculo de radio 2.6 es: 21.2264

In [12]:
```

Terminal de IPython Historial

Entrada, procesamiento y salida

Los programas de computadora realizan estos tres pasos:

1. Se recibe la **entrada**.
2. Se realiza un **proceso** sobre la entrada.
3. Se genera la **salida**



Inputs



Process

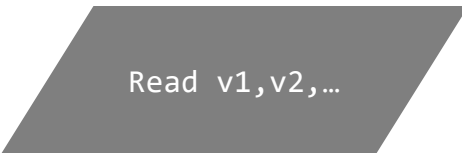
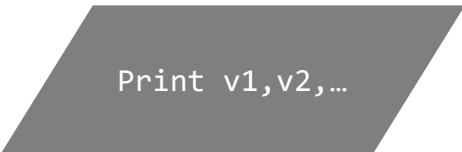


Outputs



Tabla resumen

La siguiente tabla muestra la representación equivalente de las piezas esenciales para construir código secuencial

Tipo	Diagrama de bloques	Pseudocodigo	Python
Entrada 		Leer(v1,v2,...)	<pre>v1 = input(...) v2 = input(...)</pre>
Salida 		Escribir(v1,v2,...)	<pre>print(v1,v2,...)</pre>
Proceso 		$c = a + b$	$c = a + b$

En Python, `input()` siempre devuelve texto (`str`); si el valor se va a usar en una operación numérica, debe convertirse con `int()` o `float()` (por ejemplo: `v1 = float(input(...))`).

Expresiones y operadores

Process

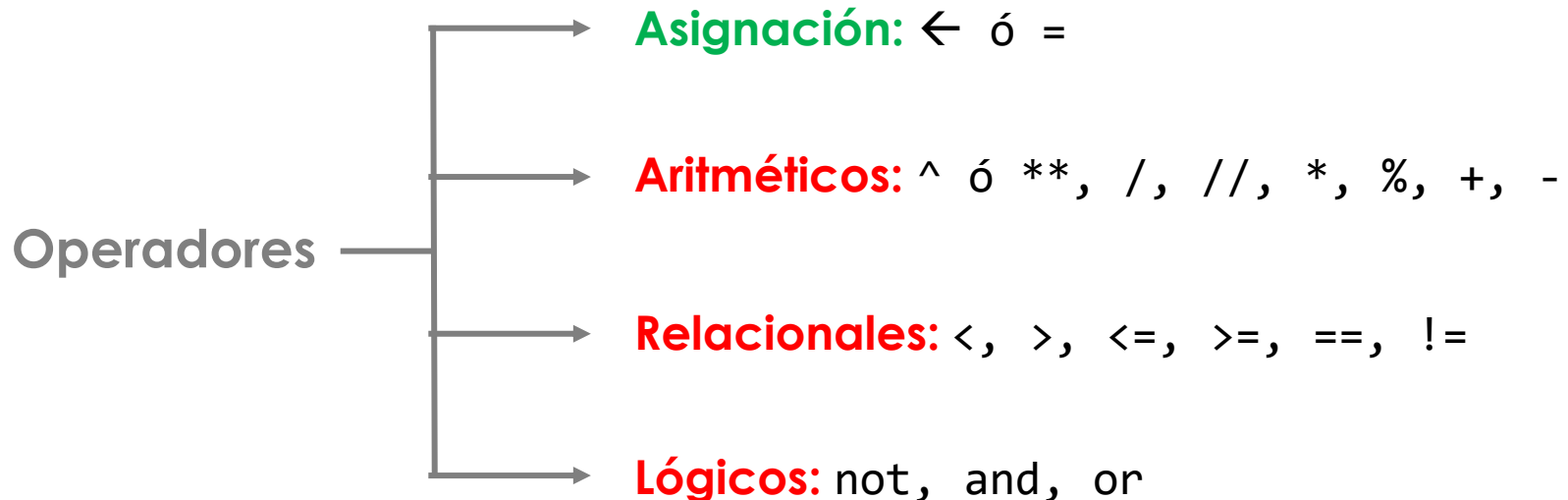


$z \leftarrow x + y$

Expresion: $z \leftarrow x + y$

- Operandos: z, x, y
- Operadores: $\leftarrow, +$

- Una **expresión** es un grupo de operadores que actúan sobre operandos para proporcionar un único resultado.



Operadores aritméticos

Los operadores aritméticos toman los valores numéricos como sus operandos y devuelve un solo valor numérico.

Operador	Nombre	Ejemplo	Resultado
+	Suma	1 + 3	4
-	Resta	7 - 10	-3
*	Multiplicación	3*5	15
/	División	8/5	1.6
//	División entera	8//3	2
%	Modulo (Residuo)	15%7	1
** ó ^	Potencia	2**3	8

Reglas de prioridad y asociatividad

Cuando concurren en una misma expresión diferentes tipos de operadores se llevan a cabo las reglas de prioridad y asociatividad.

Operación	Operador	Tipo	Asociatividad	Prioridad
Parentesis	()	----	De adentro hacia afuera	1
Potenciación	$\wedge$ ó $**$	Binario	$D \rightarrow I$	2
Identidad	+	Unitario	----	3
Cambio de signo	-	Unitario	----	
Multiplicación	*	Binario	$I \rightarrow D$	4
División	/	Binario	$I \rightarrow D$	
Módulo (residuo)	%	Binario	$I \rightarrow D$	
División entera	//	Binario	$I \rightarrow D$	
Suma	+	Binario	$I \rightarrow D$	5
Resta	-	Binario	$I \rightarrow D$	

Reglas de prioridad y asociatividad

- Al utilizar dos o más operadores en una sentencia, algunos operadores tienen precedencia sobre otros operadores.
- La **precedencia** y **asociatividad** de los operadores determina el orden en que se procesan los operadores.

Ejemplos:

Cual es el resultado de las siguientes operaciones.

a. $2 * 3 - 5 + 6$

b. $2 * (3 - 5) + 6$

c. $2 * 3 - 5 + 6/4$

d. $2 * 3 - 5 ** 2 + 1$

e. $2 * 3 - 5 ** (2 + 1)$

f. $2 ** 3 ** 2$

g. $(2 ** 3) ** 2$

h. $2 + 3 ** 2 * 5$

i. $2 + ((3 ** 2) * 5)$

Reglas de prioridad y asociatividad

Ejemplos – Solución:

a. $2 * 3 - 5 + 6$
 $6 - 5 + 6$
 $1 + 6$
 7

b. $2 * (3 - 5) + 6$
 $2 * (-2) + 6$
 $-4 + 6$
 2

c. $2 * 3 - 5 + 6/4$
 $6 - 5 + 6/4$
 $6 - 5 + 1.5$
 $1 + 1.5$
 2.5

d. $2 * 3 - 5 ** 2 + 1$
 $2 * 3 - 25 + 1$
 $6 - 25 + 1$
 $-19 + 1$
 -18

e. $2 * 3 - 5^{(2 + 1)}$
 $2 * 3 - 5^3$
 $2 * 3 - 125$
 $6 - 125$
 -119

f. $2 ** 3 ** 2$
 $2 ** 9$
 512

g. $(2 ** 3) ** 2$
 $8 ** 2$
 64

h. $2 + 3 ** 2 * 5$
 $2 + 9 * 5$
 $2 + 45$
 47

i. $2 + ((3 ** 2) * 5)$
 $2 + (9 * 5)$
 $2 + 45$
 47

Reglas de prioridad y asociatividad

- **Expresiones algebraicas:** Como lo escribimos matemáticamente.
- **Expresiones por computador:** Como debemos escribirlas para el computador.

Expresiones algebraicas

$$m = \frac{a + b + c + d + e}{5}$$



Expresiones algebraicas

$$m = (a + b + c + d + e)/5$$



$$m = a + b + c + d + e/5$$



Reglas de prioridad y asociatividad

Ejemplo:

Teniendo en cuenta los conceptos de prioridad llenar la siguiente tabla:

Expresiones algebraicas	Expresiones por computador
$\frac{\frac{x-1}{y} + 3}{\frac{z-5}{xy - 2^{x+y}}} + w$	
	$x ** 3 ** y + z + w / 1 ** (t + 3) * 5$
	$(x ** 3) ** y * (z + 1)$
$wx^{2m(n+1)} + xy^z$	

Reglas de prioridad y asociatividad

Ejemplo – solución:

Teniendo en cuenta los conceptos de prioridad llenar la siguiente tabla:

Expresiones algebraicas	Expresiones por computador
$\frac{\frac{x-1}{y} + 3}{\frac{z-5}{xy} - 2^{x+y}} + w$	$((x-1)/y + 3)/((z-5)/(x * y - 2 ** (x + y))) + w$
$x^{(3^y)} + z + \frac{w}{1^{t+3}} 5$	$x ** 3 ** y + z + w / 1 ** (t + 3) * 5$
$(x^3)^y (z + 1)$	$(x ** 3) ** y * (z + 1)$
$wx^{2m(n+1)} + x^{y^z}$	$w * x ** (2 * m * (n + 1)) + x ** y ** z$

Operadores relacionales

Un operador relacional compara sus operandos y devuelve un valor lógico (booleano) basado en si la comparación es verdad o no.

Operador	Nombre	Ejemplo	Resultado
>	Mayor	1 > 3	F
>=	Mayor o igual	2 >= 1	V
<	Menor	-5 < -1	V
<=	Menor o igual	3 <= 3	V
!=	Diferente	13 != 4	V
==	Igual (Comparación)	0 == 1	F

Operadores lógicos

- Un operador lógico toma dos operandos lógicos cada uno de los cuales es un valor true (V) o false (F) y devuelve un valor true (V) o false (F).
- Un operador lógico se utiliza para combinar múltiples comparaciones en una expresión condicional.

Operador	Nombre	Ejemplo	Resultado
not	No	not(T)	F
and	y	T and F	F
or	o	T or F	V

Not

x	not(x)
F	V
V	F

And

x	y	x and y
F	F	F
F	V	F
V	F	F
V	V	V

Or

x	y	x or y
F	F	F
F	V	V
V	F	V
V	V	V

Retornando a las reglas de prioridad

La siguiente tabla resume las reglas de prioridad teniendo en cuenta ya, todos los operadores vistos hasta el momento (aritméticos, relacionales, lógicos).

Prioridad	Operadores	Asociatividad
1	()	Adentro $\rightarrow$ Afuera
2	**	D $\rightarrow$ I
3	+ (unitario), - (unitario)	D $\rightarrow$ I
4	*, /, //, %	I $\rightarrow$ D
5	+, - (binarios)	I $\rightarrow$ D
6	>, >=, <, <=, !=, ==	I $\rightarrow$ D (encadenados)
7	not	D $\rightarrow$ I
8	and	I $\rightarrow$ D
9	or	I $\rightarrow$ D

Operadores relacionales y lógicos

Ejemplos:

Si las variables X, Z y L tienen almacenados los valores de 23, 34 y V respectivamente, ¿Cuáles serán los resultados de las siguientes expresiones?

- a. $(X > Z) \text{ or } (Z > 20)$
- b. $(X < Z) \text{ and } (\text{not}(Z > 20))$
- c. $(Z > 50) \text{ and } L$
- d. $\text{not}(L)$
- e. $(Z > 100) \text{ and } (X < 3) \text{ or } (\text{not}(L))$

Operadores relacionales y lógicos

Ejemplos - Solución:

A continuación se muestran las variables y sus valores

- $Z = 23$
- $X = 34$
- $L = V$

a. $(X > Z) \text{ or } (Z > 20)$
 $(34 > 23) \text{ or } (23 > 20)$
 $V \text{ or } (23 > 20)$
 $V \text{ or } V$
 V

b. $(X < Z) \text{ and } (\text{not}(Z > 20))$
 $(34 < 23) \text{ and } (\text{not}(23 > 20))$
 $(34 < 23) \text{ and } (\text{not}(V))$
 $(34 < 23) \text{ and } (F)$
 $(F) \text{ and } (F)$
 F

Operadores relacionales y lógicos

Ejemplos - Solución:

A continuación se muestran las variables y sus valores

- $Z = 23$
- $X = 34$
- $L = V$

c. $(Z > 50) \text{ and } L$
 $(23 > 50) \text{ and } V$
 $F \ \& \ V$
 F

d. $\text{not}(L)$
 $\text{not}(V)$
 F

e. $(Z > 100) \text{ and } (X < 3) \text{ or } (\text{not}(L))$
 $(23 > 100) \text{ and } (24 < 3) \text{ or } (\text{not}(V))$
 $(23 > 100) \text{ and } (24 < 3) \text{ or } F$
 $F \text{ and } (24 < 3) \text{ or } F$
 $F \text{ and } F \text{ or } F$
 $F \text{ and } F$
 F

Ejercicios de repaso

1. Escriba las siguientes expresiones algebraicas como expresiones algorítmicas:

a. $x^2 + 4xyz$

b. $\frac{x+y}{z} + \frac{3x}{5} + 4y$

c. $\frac{-b + \sqrt{b^2 - 4ac}}{2a}$

d. $\frac{4x^2 - 2x + 8}{c - d}$

2. Escriba las siguientes expresiones algebraicas como expresiones algorítmicas:

a. $b ** 2 - x * y$

b. $x * y - y ** (1/2)$

c. $(((B + C) / 2 * A + 10) * 3 * B) - 6$

Ejercicios de repaso

3. **(Repaso)** Cuales de los siguientes identificadores no son validos

- a. XRayo*
- b. X_Rayo*
- c. R2D2*
- d. X*
- e. 45*
- f. N14*
- g. ZZZZ*
- h. 3u*

4. Si el valor de A es 2, el de B es 3 y el de C es 2, evaluar la siguiente expresión:

$$A ** B ** C$$

Ejercicios de repaso

5. Si el valor de A es 4, el de B es 5 y el de C es 1, evaluar las siguientes expresiones:
- a. $B * A - B ** 2 / 4 * C$
 - b. $(A * B) / 3 ** 2$
 - c. $(((B + C) / 2 * A + 10) * 3 * B) - 6$
6. Obtener el valor de cada una de las siguientes expresiones aritméticas:
- a. $7 // 2$
 - b. $7 \% 2$
 - c. $12 // 3$
 - d. $0 \% 5$
 - e. $15 \% 5$
 - f. $7 * 10 - 50 \% 3 * 4 + 9$
 - g. $(7 * (10 - 50) \% 3) * 4 + 9$

Aplicando lo anterior a un lenguaje de programación específico

- Lo que vimos hoy es la convención más extendida (herencia de la notación matemática y de C).
- Pero prioridad y asociatividad son decisiones de diseño de cada lenguaje, no leyes fijas.
- Ante la duda: paréntesis explícitos, o consultar la documentación oficial del lenguaje.



Operadores lógicos — comparación entre lenguajes

El mismo concepto lógico, escrito distinto según el lenguaje:

- Cada lenguaje escribe "y", "o", "no" lógico con su propia sintaxis: no son intercambiables entre lenguajes.
- `&` y `|` también existen, pero son otra cosa (bit a bit): otra prioridad, y no hacen cortocircuito.

Concepto	Python	Java / C / JavaScript
AND lógico (con cortocircuito)	<code>and</code>	<code>&&</code>
OR lógico (con cortocircuito)	<code>or</code>	<code> </code>
NOT lógico	<code>not</code>	<code>!</code>
AND / OR bit a bit (<i>no confundir con los anteriores</i>)	<code>&</code> / <code> </code>	<code>&</code> / <code> </code>

- **(Pregunta)** ¿Qué pasa cuando se usa `&` en vez de `and` dentro de una comparación sin paréntesis?

```
x = 6, y = 3
z = x > 5 and y < 3
```

```
x = 6, y = 3
z = x > 5 & y < 3
```

Igualdad — comparación entre lenguajes

El mismo símbolo `==`, comportamientos distintos:

- En Python, `==` compara el **valor/contenido**, como uno esperaría intuitivamente.
- En C y Java, `==` compara valor en tipos primitivos, pero **dirección/referencia** en strings u objetos, no el contenido.
- En JavaScript, `==` compara **con conversión automática de tipos**; `===` compara sin conversión (la forma recomendada).

Lenguaje	Operador	Qué compara
Python	<code>==</code>	Valor/contenido
Python	<code>is</code>	Identidad (misma referencia)
C	<code>==</code>	Valor (tipos primitivos) / Dirección (strings)
C	<code>strcmp()</code>	Contenido (strings) — 0 significa "iguales"
Java	<code>==</code>	Valor (primitivos) / Referencia (objetos)
Java	<code>.equals()</code>	Valor/contenido (objetos)
JavaScript	<code>==</code>	Valor, con conversión de tipos
JavaScript	<code>===</code>	Valor, sin conversión

Potenciación — comparación entre lenguajes

El símbolo $\wedge$ no significa lo mismo en todas partes:

- En Python, C, Java y JavaScript, $\wedge$ es **XOR** bit a bit, no potencia.
- Python usa ****** para potencia; C y Java no tienen operador de potencia (se usa una función).
- Excel, MATLAB y algunas calculadoras sí usan $\wedge$ como potencia — de ahí viene la confusión.

Lenguaje	Potencia	$\wedge$ significa
Python	**	XOR bit a bit
C	pow() (función, #include <math.h>)	XOR bit a bit
Java	Math.pow() (función)	XOR bit a bit
JavaScript	** (o Math.pow())	XOR bit a bit
Excel / MATLAB	$\wedge$	Potencia

- **(Pregunta)** Si escribe **5 $\wedge$ 2** en Python esperando "5 al cuadrado", ¿qué cree que se va a devolver realmente?

Asociatividad — comparación entre lenguajes

La forma en que se agrupan operadores repetidos también cambia:

- La potenciación asocia de derecha a izquierda en Python ($2^{**}3^{**}2 = 2^{**}(3^{**}2)$), pero no todos los lenguajes la implementan igual, y C/Java ni siquiera tienen `**`.
- Las comparaciones encadenadas ($a < b < c$) son un caso especial en Python; en Java, C y JavaScript no existe esa sintaxis de la misma forma, y si se escribe algo parecido, el resultado es distinto.
- La asignación encadenada ($a = b = c$) funciona en Python, C, Java y JavaScript, pero en Python la asignación no es una expresión (es una sentencia), mientras que en los otros sí lo es.

Caso	Python	Java / C / JavaScript
$a < b < c$	Cadena especial: equivale a $(a < b) \text{ and } (b < c)$	No existe igual; se evalúa distinto (comparación entre bool y c)
$a = b = c$	Válido, pero <code>=</code> es sentencia, no expresión	Válido, <code>=</code> es expresión, asocia D→I
$2^{**}3^{**}2$	Asocis de D→I es decir da 512	No aplica (Java/C sin <code>**</code>); en JS si aplica, también D→I

Referencias adicionales — tablas de precedencia por lenguaje

Dado que cada lenguaje define sus propias reglas de precedencia, consulte la fuente oficial correspondiente ante cualquier duda:

1. **Python — Operator precedence (documentación oficial):**
<https://docs.python.org/3/reference/expressions.html#operator-precedence>
2. **Java — Java Language Specification, cap. 15 (Oracle):**
<https://docs.oracle.com/javase/specs/jls/se21/html/jls-15.html>
3. **C — Operator precedence (cppreference):**
https://en.cppreference.com/w/c/language/operator_precedence
4. **JavaScript — Operator precedence (MDN):** https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Operator_precedence

Referencias

1. <https://ellibrodepython.com/>
2. <https://www.w3resource.com/index.php>
3. <https://realpython.com/>
4. <https://curriculumresources.edu.gh/wp-content/uploads/2025/01/>
5. <https://python-course.eu/>
6. <https://realpython.com/cheatsheets/python/>

Eso es todo amigos



Number operations.